

DATABASE MANAGEMENT SYSTEM

24B15CS213



JAYPEE INSTITUTE OF INFORMATION TECHNOLOGY

SECTOR 128, NOIDA

Movie Production & Film Industry Database Management System

TEAM MEMBERS: -

RACHNA 992401030362

ANUSHKA JAIN 992401030363

SUBMITTED TO:

DR. ADITI SHARMA

Acknowledgement

We would like to express our heartfelt gratitude to our mentor Dr. Aditi Sharma, for her continuous guidance, support, and valuable feedback throughout this project. Her insights and encouragement were instrumental in the successful completion of our project titled “Movie Production & Film Industry Database Management System”.

We would also like to thank Jaypee Institute of Information Technology for providing us with the necessary resources and a collaborative learning environment. Lastly, we thank our teammates for their cooperation, hard work, and dedication throughout the development process.

Certificate of Completion

This is to certify that the project titled "Movie Production & Film Industry Database Management System".has been successfully completed by the following students as part of their B.Tech Semester 3 coursework under the supervision of Dr.Aditi Sharma, Faculty,Department of Computer Science & Engineering, IIIT, Noida.

Rachna(992401030362)

Anushka Jain(992401030363)

Faculty Guide: Dr. Aditi Sharma

Department of Computer Science & Engineering

Jaypee Institute of Information Technology, Sector-128, Noida

1. Introduction

In today's digital era, the film and entertainment industry relies heavily on data-driven platforms to manage movies, cast members, production studios, awards, and releases across multiple channels. This project, **Movie Database Management System**, focuses on developing an efficient database using **MySQL and core DBMS concepts**. The objective of the system is to store and manage extensive film-related information while ensuring accuracy, consistency, and fast data retrieval.

The system maintains details of **movies, actors, directors, producers, studios, OTT platforms, shooting locations, and awards**, along with their inter-relationships. The database supports operations such as inserting new movies, updating information, assigning actors to a movie, tracking awards won, linking OTT releases, managing production teams, and viewing analytical reports such as highest-rated or most-awarded movies.

To ensure industrial-standard database reliability, the project has been implemented with **strong constraints**, fulfilling the core requirements like an ER-based relational database, SQL queries, and complex PL/SQL logic

2. Problem Statement

Managing information in the film and entertainment industry involves multiple interconnected entities such as movies, actors, directors, producers, studios, awards, and OTT releases. In the absence of a structured database, storing and retrieving such data becomes tedious, error-prone, and inconsistent. Production houses, media analysts, and streaming platforms require a system capable of handling detailed movie information, cast and crew mapping, box-office performance, award tracking, and digital release records efficiently.

This project addresses the need for a structured and reliable Movie Database Management System by designing a relational database using MySQL that accurately represents real-world relationships in the entertainment domain. The system stores complete information about movies and their associated elements — including cast, crew, shooting locations, award history, and OTT availability — while enforcing data integrity through primary keys, foreign keys, check constraints.

3. Proposed Solution / Methodology

3.1 System Architecture

1. Database Layer (Primary Focus)

- MySQL relational schema consisting of core entities: Person, Movie, Actor, Director, Producer, Studio, OTTPlatform, Location, Award, Employee, Manager, Technician, and junction tables such as Movie_Actor, Movie_Producer, Movie_Location, Movie_Award, and Movie_OTT.
- Proper use of **primary keys, foreign keys, unique constraints, and check constraints** to maintain referential integrity and valid data ranges.

2. PL/SQL Logic Layer

- **Stored Functions** such as fn_GetMovieAge and fn_CalcAwardPoints implement reusable calculations for movie age and award points.
- **Stored Procedures** (e.g., AddMovieBasic, SearchMovie, UpdateMoviePerformance, AddMovieAward) encapsulate insertion, search, and update logic, including error handling in selected procedures using SQL exception handlers.
- **Cursor-based Procedures** (CreateOttMovieList, SumBudgetByStudioCursor) sequentially process result sets to build aggregated outputs like OTT movie lists and total studio budget.
- **Triggers** on Movie, Movie_Award, and related tables enforce business rules such as validating award year, logging rating changes, maintaining audit tables, and updating award points automatically.

3. Application / UI Layer (PHP on XAMPP)

- PHP pages hosted on XAMPP connect to the MySQL database using standard connectors (e.g., mysqli/PDO).
- The UI provides basic options such as **Add Movie, View Movies, Search Movie, Update Movie, and Delete Movie**, which internally call the corresponding SQL queries and stored procedures.
- Result sets from queries, procedures, and cursor output are displayed in tabular form on the web interface to demonstrate real-time interaction with the database.

3.2 Implementation Steps

1. Requirement Analysis & Domain Study

Identify essential entities in the movie ecosystem: movies, cast, crew, studios, OTT platforms, locations, and awards, along with their relationships.

2. ER Modeling & Conversion to Tables

- Draw an ER diagram capturing relationships like *Movie–Actor*, *Movie–Producer*, *Movie–OTTPlatform*, and *Movie–Award*.
- Convert the ER diagram into relational tables with appropriate keys and constraints.

3. Table Creation in MySQL (XAMPP)

- Implement CREATE TABLE statements for all entities and relationship tables using data types and constraints consistent with course guidelines.
- Execute these scripts in phpMyAdmin / MySQL console under XAMPP and verify successful creation.

4. Sample Data Insertion

- Populate each table with meaningful sample records (e.g., “Avengers: Endgame”, “Interstellar”, “Dune: Part One”, real actors, studios, OTT platforms, and awards).
- Use INSERT statements following the syntax taught in DBMS lectures to enable realistic query outputs.

5. Design of SQL Queries

- Implement at least **15 SQL queries** covering:
- simple SELECT with WHERE, ORDER BY, LIKE, BETWEEN, IN multi-table **joins** to show movies with actors, studios, OTT platforms, and awards **grouping and aggregation** (e.g., total box-office per studio, average rating by genre)**nested subqueries**, EXISTS, ANY/ALL to answer analytical questions such as movies with maximum box office or actors in Netflix movies.

6. Development of Functions, Procedures, and Cursors

- Create functions for domain-specific calculations (movie age, award points).
- Develop procedures for reusable operations such as inserting new movies, updating performance, assigning awards, and searching by title/genre.

- Implement cursor-based procedures to iterate over result sets and generate summary outputs (OTT movie list, total budget).

7. Trigger Implementation

Add triggers to:

- validate award years against movie release years,
- log new movie insertions into an audit table,
- maintain cumulative award points for each movie,
- track rating changes and movie deletions in separate log tables.

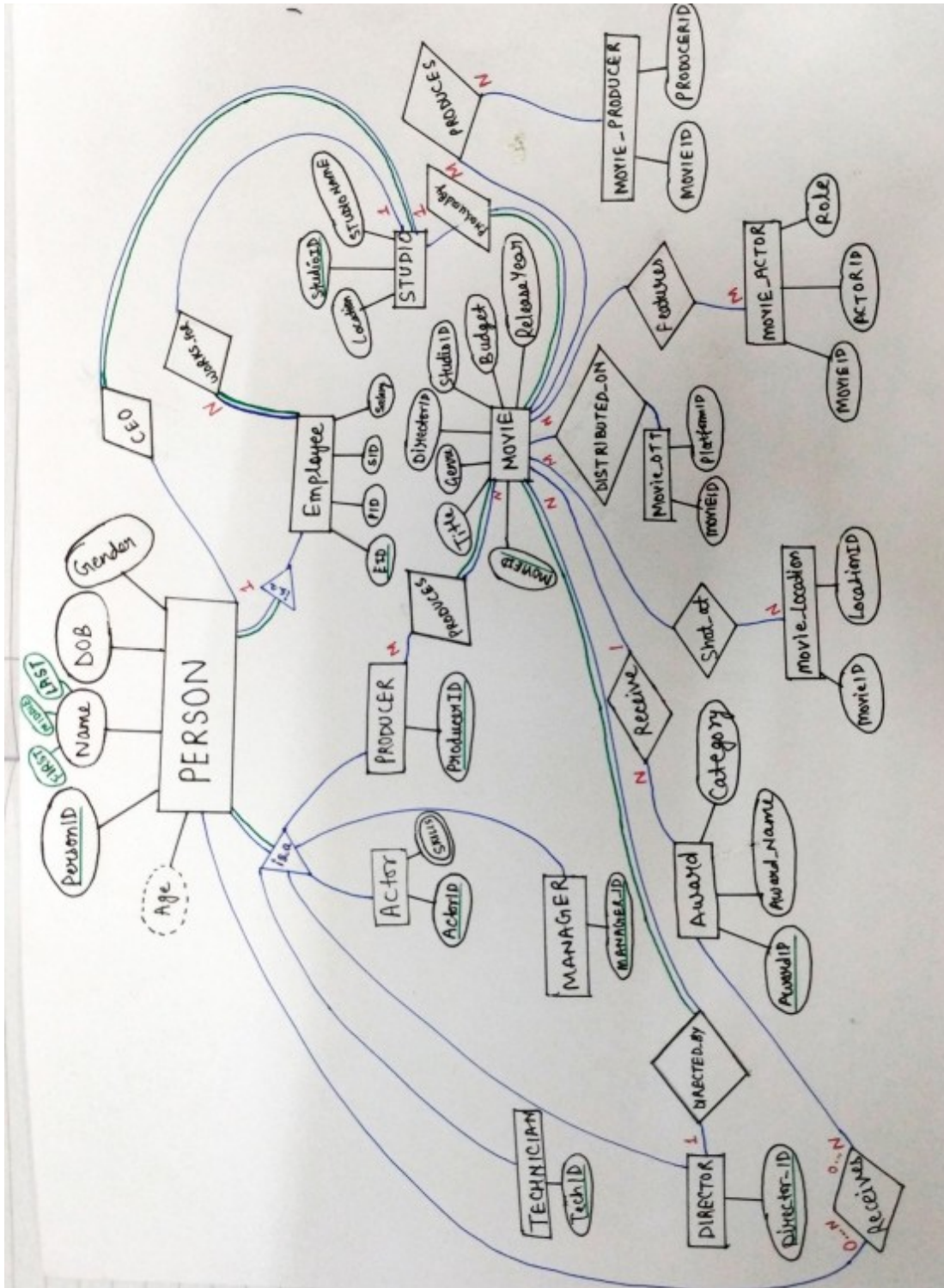
8. PHP Integration & Testing

- Connect the PHP front-end to MySQL using XAMPP, map UI buttons to procedures/queries, and render results in HTML tables.
- Test each module (insertion, querying, function/procedure calls, cursor outputs, and trigger behavior) using the web interface and phpMyAdmin to ensure correctness and data integrity.

3.3 DBMS Concepts Demonstrated

- **Relational Modeling & Normalization** through properly structured tables and relationships.
- **Constraints & Keys** to enforce entity and referential integrity.
- **Joins, Aggregates, and Subqueries** for complex analytical retrieval.
- **Stored Functions, Procedures, Cursors, and Triggers** to implement business logic at the database level with selected exception handling.
- **Web-based Integration** using PHP and XAMPP to show how SQL and PL/SQL components are used in a real application.

4. ER Diagram



5. Expected Outcome:

A functional mini **Movie Database Management System** where all core features—movie registration, cast and crew mapping, studio and OTT assignment, award tracking, and basic update/delete operations—are fully managed through MySQL and accessed via a PHP interface on XAMPP.

A clear understanding of **database design, normalization up to 3NF, relational mapping, and SQL operations** as applied to a realistic film and entertainment domain.

Practical implementation of complex relationships between **movies, actors, directors, producers, studios, locations, OTT platforms, and awards** using primary keys, foreign keys, check constraints, and different types of joins in at least fifteen SQL queries.

Hands-on experience with **PL/SQL components**—stored functions, stored procedures (with error handling), cursors, and triggers—to automate calculations, validations, logging, and summary generation directly at the database level.

A modular, well-structured **database backend that can be easily extended** with additional PHP pages or any other web framework to build richer movie analytics and user-facing features in the future.

A realistic demonstration of how professional movie platforms manage and process data such as film details, cast lists, box-office figures, OTT releases, and awards at the **database level** while maintaining integrity and consistency.

6.Conclusion

The **Movie Database Management System** project successfully demonstrates how core entertainment-domain data operations can be implemented through **structured database design using MySQL**. By building a well-organized relational schema with clearly defined tables, relationships, constraints, and optimized SQL operations, the system achieves maintainability, scalability, and clarity in storing and managing interconnected movie-related information.

This project helped us understand essential DBMS concepts such as relational movie management, cast and crew mapping, OTT and award tracking, data retrieval through joins and grouping, and backend automation through PL/SQL features. Implementing more than fifteen structured SQL queries, along with **stored functions, stored procedures (with exception handling), cursors, and triggers**, strengthened our understanding of **normalization, constraints, integrity enforcement, PL/SQL programming, and relational mapping**, reflecting how real-world movie platforms store and process large-scale entertainment data at the database level.

7.References

- **MySQL Documentation** – Official reference manual for SQL syntax, table creation, constraints, functions, procedures, cursors, and triggers.
- **Elmasri & Navathe, *Fundamentals of Database Systems*** – Concepts of relational modeling, normalization, ER-to-table mapping, keys, and constraints.
- **W3Schools – MySQL Tutorial** – SQL queries, joins, grouping, subqueries, and PHP–MySQL connectivity basics.
- **GeeksforGeeks – DBMS & SQL Articles** – Normalization rules, relational mapping, and real-world database examples.
- **TutorialsPoint – MySQL PL/SQL Programming** – Stored functions, stored procedures, cursor operations, and trigger implementation.
- **XAMPP & PHP Documentation** – Guidelines for establishing database connectivity, executing SQL operations from PHP, and displaying dynamic results in the UI.
- **StackOverflow Developer Discussions** – Community-driven clarifications on SQL query optimization and PL/SQL debugging during implementation.